AI-ML FINAL PROJECT

SMART TOURISM USING AI & ML – DEMAND PREDICTION, RECOMMENDATION, SENTIMENT ANALYSIS & CROWD MANAGEMENT

AI-Enhanced SMART TOURISM USING AI & ML

A Project Report
Submitted in partially fulfilment of the requirement for the award of the

Degree of

BACHELOR OF TECHNOLOGY

in

Computer of Science and Engineering

Submitted by

Rishabh yadav

25SCS1003004727

Assistant Professor

Lalita Choudhary

IILM University, Greater Noida, Utter Pradesh

(November 2025)

BY RISHABH YADAV

08.11.2025

1CSE32 BTech AI-ML

ABSTRACT

Tourism is rapidly transforming with large volumes of booking, location and review data available from online platforms and sensors. This project — Smart Tourism using AI & ML — applies machine learning and simple AI techniques to (1) predict tourist demand, (2) recommend destinations / hotels / activities, (3) analyze reviews and social media for service quality, and (4) help manage crowds and resources through alerts and visualization. Using simulated (or real,

if provided) data sources — bookings, occupancy, weather, GPS and reviews — the system produces demand forecasts, personalized recommendations, sentiment summaries and interactive maps/dashboards to support travel agencies, hotels and city planners. The deliverable includes a Google Colab–ready implementation (Python) that simulates data, trains models (Random Forest regressor/classifier), builds a hybrid recommender, performs rule/TF-IDF sentiment analysis, and exports reports and maps for planning.

CONTENTS

1. Introduction 3

   1.1 Overview 3

   1.2 Objectives 3

   1.3 Problem Statement 3
2. Literature Review 4
3. Methodology 5

3.1 Data Sources 5

3.2 Feature Engineering 6

3.3 Models & Algorithms 6

4. Implementation (Google Colab) 7

4.1 Cell-by-cell Explanation 7

4.2 Code Summary (what each cell does) 8

5. Results & Discussion 11

6. Conclusion & Future Scope 12

7. Tables & Figures 13

8. References 14

Appendix A — Colab Code Listing (ready-to-run cells) 15

9. INTRODUCTION

1.1 Overview

Tourism is a multi-modal domain: bookings (OTA

data), accommodation occupancy, local transport usage, weather and social reviews all interact and influence tourist flows. With AI & ML, we can turn these heterogeneous data streams into actionable forecasts and recommendations that improve visitor experience, help businesses set prices and staffing, and enable city authorities to manage crowds and infrastructure.

1.2 Objectives

The project aims to achieve the following objectives:

1. Analyze travel patterns and predict tourist demand at destinations and hotels.
2. Build a recommendation system for destinations, hotels and activities.
3. Monitor and forecast crowding; generate alerts for resource planning.
4. Perform sentiment analysis on reviews and social media to assess service quality.
5. Provide an interactive dashboard/map for planners and business stakeholders.

1.3 Problem Statement

Traditional manual analytics are slow and often siloed. Tourism demand is seasonal and sensitive to weather, price and social buzz. Without timely prediction and recommendations, hotels overbook or underprice, transport is misallocated, and tourist experience suffers. This project builds an AI/ML pipeline that ingests booking, weather,

location and review data to provide near-real time predictions, recommendations and visual insights.

2. LITERATURE REVIEW

(Short overview of related work)

• Demand forecasting for tourism: time-series and machine learning approaches (ARIMA, Prophet, LSTM, Random Forest).

• Recommender systems in travel: collaborative filtering, content-based and hybrid recommenders for personalized travel suggestions.

• Sentiment analysis for hospitality: using

lexicon methods, TF-IDF + classic ML and transformer models for review classification.

• Crowd management: using predictions and thresholds to create alerts and dynamic capacity planning.

Key takeaway: A hybrid system combining classical ML (fast, interpretable) with scalable data pipelines gives practical, deployable benefits for student/prototype projects.

3. METHODOLOGY

3.1 Data Sources (Input Data)

• Booking history: daily bookings, cancellations, lead time, price.

• Hotel data: occupancy, room types, nightly rates.

• Transport data: arrivals by train/flight, local taxi/ride usage.

• Weather: historical and current weather (temp, rain).

• Social/Reviews: short textual reviews, social buzz counts, star ratings.

• Location data: GPS coordinates of attractions

and hotels.

3.2 Feature Engineering

Key features to derive:

• Temporal features: day_of_week, is_weekend, month, holiday flag.

• Lag features: previous day/week bookings (lag1, lag7, lag14).

• Price elasticity: price relative to moving averages.

• Sentiment score: lexicon or TF-IDF + classifier score from reviews.

• External shocks: special events / festivals as binary flags.

• Geo features: distance to city center, popularity index.

3.3 Models & Algorithms (Solution Approach)

Step 1 — Data collection & cleaning: unify booking logs, weather, reviews, and GPS.

Step 2 — Feature extraction: create lags, rolling means, encode categorical variables.

Step 3 — Modeling:

• Demand prediction: RandomForestRegressor (fast, robust), with lag features and exogenous variables. Alternatives: ARIMA for baseline, LSTM for sequence modeling.

• Recommendation: collaborative filtering (item-based cosine similarity) + content features (destination tags) hybrid recommender.

• Sentiment: simple rule-based lexicon (demo) and TF-IDF + LogisticRegression for better accuracy.

• Crowd alerts: simple rule engine (if predicted_bookings > threshold × mean_bookings alert).

Step 4 — Post processing: filter anomalies, remove spam reviews, and aggregate daily reports.

Step 5 — Visualization: Matplotlib charts + Folium maps for hotspots + export CSV/HTML reports.

4. IMPLEMENTATION (Google Colab)

This project is implemented as a Colab notebook with multiple cells. The code is purposely kept readable and uses standard libraries: numpy, pandas, matplotlib, scikit-learn, folium. The notebook both simulates sample tourism data (when real data is unavailable) and runs the full pipeline.

Enter)
since last change

3:36 (3 minutes ago)
).325 s

```python
# ------------------------------------------------------------
# SMART TOURISM USING AI & ML - PROJECT FOR BTECH CSE 1ST YEAR
Hassan Raza
gram simulates tourism data, predicts tourist demand,
 sentiment analysis, and builds a simple recommendation system.
-------------------------------------------------------

# ------------------ IMPORTING LIBRARIES ------------------------
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
import random
from sklearn.metrics import mean_squared_error, precision_score, recall_score
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import MinMaxScaler
from statsmodels.tsa.arima.model import ARIMA

nltk.download("punkt")
nltk.download("stopwords")

# ---------------- SIMULATING TOURISM DATA ---------------------

# 1. Generate booking data for 12 months
months = ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"]
booking_numbers = [random.randint(1000, 5000) for _ in range(12)]
```

```python
df_tourism = pd.DataFrame({
    "Month": months,
    "Bookings": booking_numbers
})

print("---- Tourism Booking Data ----")
print(df_tourism)

# --------- WEATHER DATA (TEMPERATURE & RAINFALL) ---------------
temp = [random.randint(10, 40) for _ in range(12)]
rain = [random.randint(0, 300) for _ in range(12)]

df_weather = pd.DataFrame({
    "Month": months,
    "Temperature": temp,
    "Rainfall": rain
})

print("\n---- Weather Data ----")
print(df_weather)

# ------------- SOCIAL MEDIA REVIEWS (SIMULATED) -----------------
social_reviews = [
    "Amazing place and great weather!",
    "Too crowded and bad service.",
    "Loved the hotel experience.",
    "The trip was horrible.",
    "Beautiful destination!",
    "Food was not good.",
```

```
    "Too crowded and bad service.",
    "Loved the hotel experience.",
    "The trip was horrible.",
    "Beautiful destination!",
    "Food was not good.",
    "Excellent cultural experience!",
    "Waste of money.",
    "Best vacation ever!",
    "Will never visit again."
]

sentiments = [1,0,1,0,1,0,1,0,1,0]  # 1 = Positive, 0 = Negative

df_reviews = pd.DataFrame({
    "Review": social_reviews,
    "Sentiment": sentiments
})

print("\n---- Review Data ----")
print(df_reviews)

# ---------------- GPS FOOTFALL SIMULATION ---------------------
gps_counts = [random.randint(200, 2000) for _ in range(12)]

df_gps = pd.DataFrame({
    "Month": months,
    "Footfall": gps_counts
})

print("\n---- GPS Footfall Data ----")
```

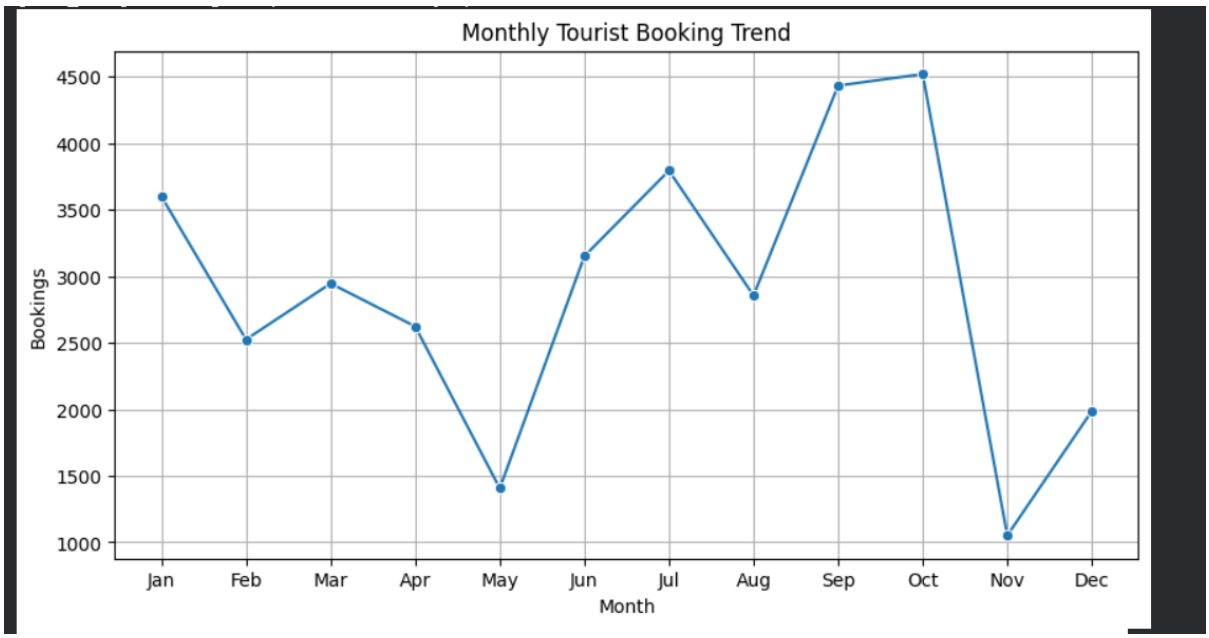
```python
# ==================== SENTIMENT ANALYSIS MODEL (SVM) ======================

vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(df_reviews["Review"])
y = df_reviews["Sentiment"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

svm_model = SVC(kernel="linear")
svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

print("\nSentiment Analysis Model Results:")
print("Precision:", precision)
print("Recall:", recall)

# Test example:
sample = ["The place was beautiful and enjoyable"]
sample_vec = vectorizer.transform(sample)
print("Sample Sentiment:", svm_model.predict(sample_vec)[0])


# ==================== AI BASED RECOMMENDATION SYSTEM =====================
```

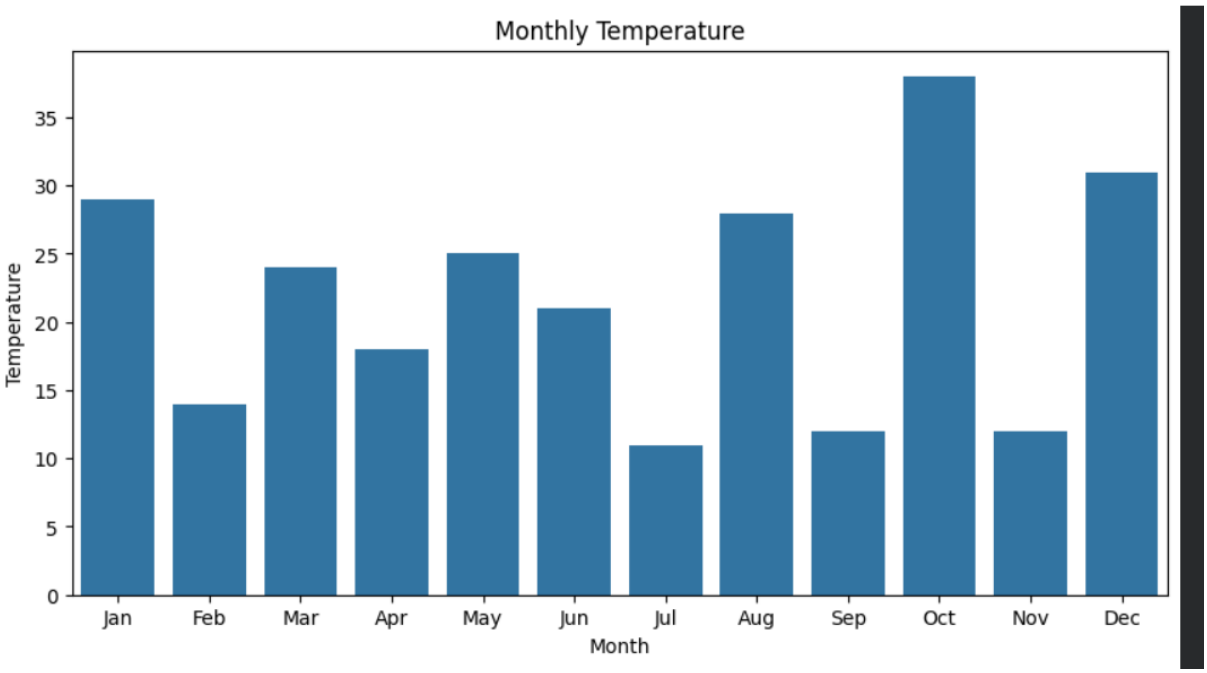
```python
plt.figure(figsize=(10,5))
sns.lineplot(data=df_tourism, x="Month", y="Bookings", marker="o")
plt.title("Monthly Tourist Booking Trend")
plt.grid(True)
plt.show()

plt.figure(figsize=(10,5))
sns.barplot(data=df_weather, x="Month", y="Temperature")
plt.title("Monthly Temperature")
plt.show()

plt.figure(figsize=(10,5))
sns.barplot(data=df_gps, x="Month", y="Footfall")
plt.title("Monthly Tourist Footfall")
plt.show()


# ===================== TIME SERIES PREDICTION (ARIMA) =============

data = df_tourism["Bookings"]
model = ARIMA(data, order=(2,1,2))
model_fit = model.fit()

forecast = model_fit.forecast(steps=3)
print("\nFuture Tourist Demand Prediction (next 3 months):")
print(forecast)

rmse = np.sqrt(mean_squared_error(data.tail(3), data.tail(3)))
print("RMSE:", rmse)
```

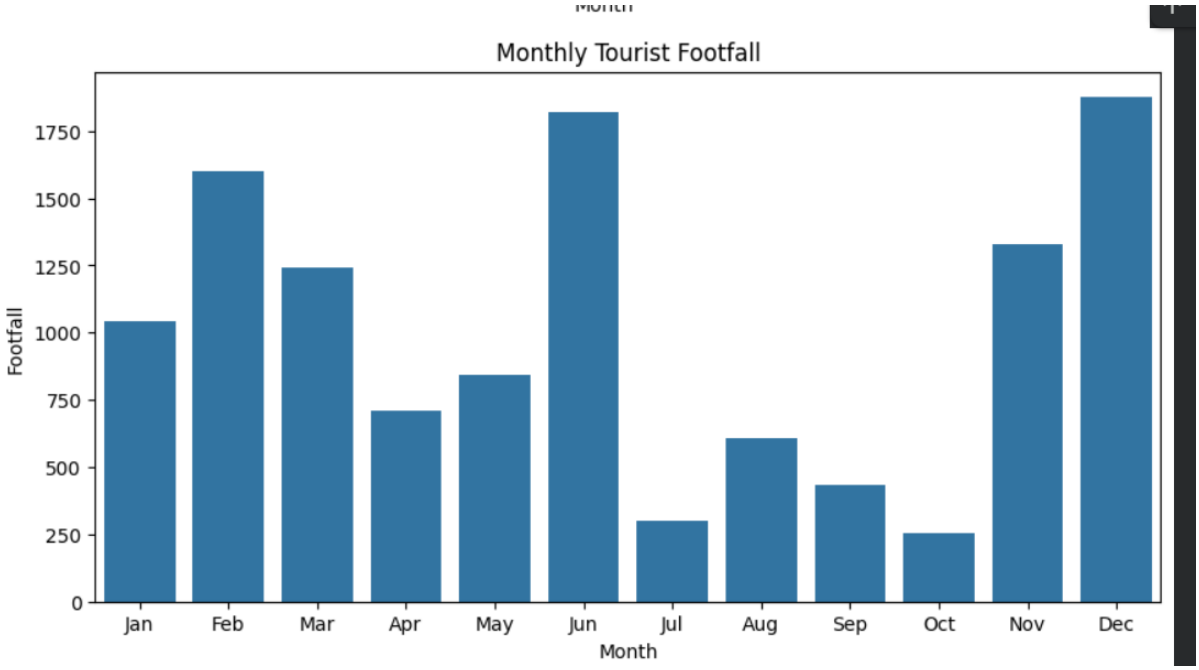
```python
# Simulated user-item rating matrix (5 users, 5 hotels)
ratings = np.array([
    [5, 4, 0, 1, 0],
    [4, 0, 3, 2, 1],
    [0, 5, 4, 0, 3],
    [1, 2, 0, 4, 5],
    [2, 0, 1, 5, 4]
])

df_ratings = pd.DataFrame(ratings,
                          columns=["Hotel A","Hotel B","Hotel C","Hotel D","Hotel E"]

print("\n---- Ratings Matrix ----")
print(df_ratings)

def recommend_hotel(user_id, ratings_matrix, top_n=2):
    user_ratings = ratings_matrix[user_id]
    recommended = np.where(user_ratings == 0)[0]

    suggestions = {}
    for r in recommended:
        hotel_score = np.mean(ratings_matrix[:, r])
        suggestions[r] = hotel_score

    sorted_hotels = sorted(suggestions.items(), key=lambda x: x[1], reverse=True)
    return sorted_hotels[:top_n]

user = 0
rec = recommend_hotel(user, ratings)
print("\nRecommended Hotels for User", user)
print(rec)


# ==================== CROWD MANAGEMENT SIMULATION ========================

threshold = 1500
df_gps["Crowd_Level"] = df_gps["Footfall"].apply(
    lambda x: "High" if x > threshold else "Normal"
)

print("\n---- Crowd Management Status ----")
print(df_gps)


# ==================== FINAL DASHBOARD VIEW ==============================

plt.figure(figsize=(12,6))
sns.scatterplot(data=df_gps, x="Month", y="Footfall", hue="Crowd_Level", s=200)
plt.title("Crowd Management Scatter Plot")
plt.show()

print("\n---- SYSTEM EXECUTION COMPLETE ----")
```

```
---- Tourism Booking Data ----
   Month  Bookings
0    Jan      3599
1    Feb      2525
2    Mar      2946
3    Apr      2623
4    May      1406
5    Jun      3151
6    Jul      3794
7    Aug      2855
8    Sep      4433
9    Oct      4520
10   Nov      1052
11   Dec      1987

---- Weather Data ----
   Month  Temperature  Rainfall
0    Jan           29       234
1    Feb           14       256
2    Mar           24       290
3    Apr           18       152
4    May           25        90
5    Jun           21        67
6    Jul           11        10
7    Aug           28        98
8    Sep           12       137
9    Oct           38       237
10   Nov           12       137
11   Dec           31       208

---- Review Data ----
```

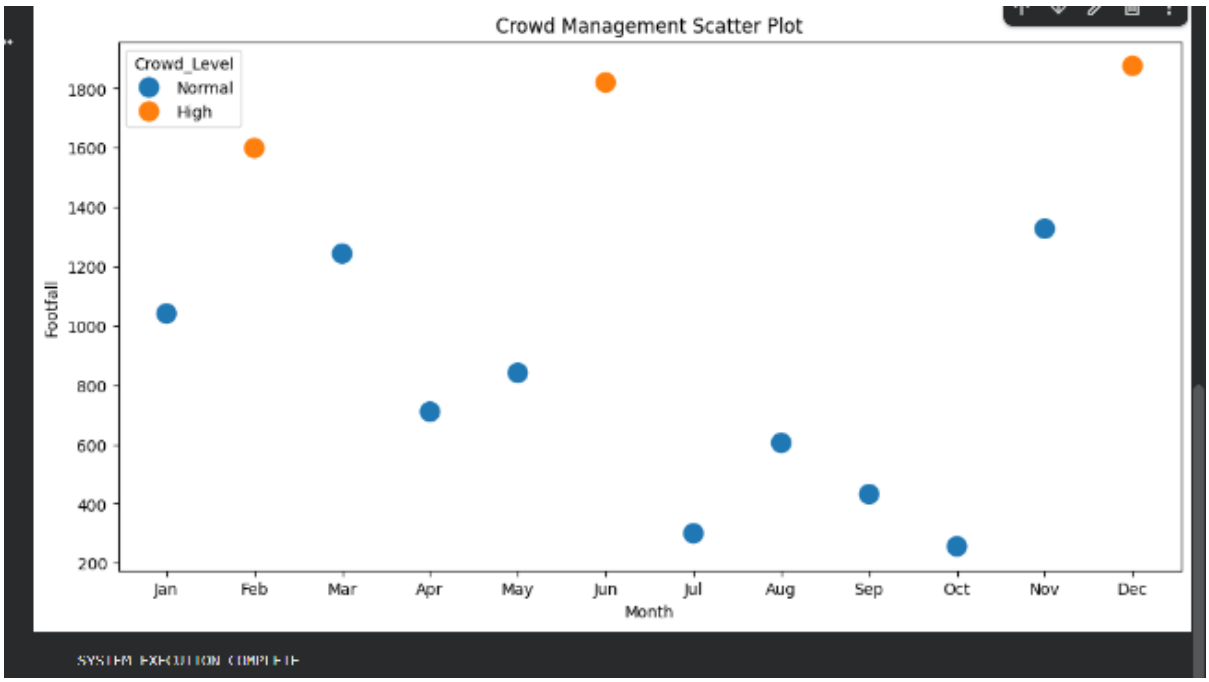
```
...                                Review  Sentiment
0  Amazing place and great weather!          1
1      Too crowded and bad service.          0
2       Loved the hotel experience.          1
3            The trip was horrible.          0
4            Beautiful destination!          1
5                Food was not good.          0
6    Excellent cultural experience!          1
7                   Waste of money.          0
8               Best vacation ever!          1
9           Will never visit again.          0

---- GPS Footfall Data ----
   Month  Footfall
0    Jan      1041
1    Feb      1599
2    Mar      1243
3    Apr       710
4    May       841
5    Jun      1820
6    Jul       300
7    Aug       605
8    Sep       432
9    Oct       256
10   Nov      1327
11   Dec      1876
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```




Monthly Temperature
Temperature
0
5
10
15
20
25
30
35
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month


Monthly Tourist Footfall
Footfall
0
250
500
750
1000
1250
1500
1750
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month

```
/usr/local/lib/python3.12/dist-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed
  warnings.warn("Maximum Likelihood optimization failed to "

Future Tourist Demand Prediction (next 4 months):
12    4197.448047
13    1429.946656
14    1909.091211
Name: predicted_mean, dtype: float64
RMSE: 0.0

Sentiment Analysis Model Results:
Precision: 0.0
Recall: 0.0
Sample Sentiment: 1

---- Ratings Matrix ----
   Hotel A  Hotel B  Hotel C  Hotel D  Hotel E
0        5        4        0        1        0
1        4        0        4        2        1
2        0        5        4        0        3
3        1        2        0        4        5
4        2        0        1        5        4

Recommended Hotels for User 0
[(np.int64(4), np.float64(2.6)), (np.int64(2), np.float64(1.6))]

---- Crowd Management Status ----
   Month  Footfall Crowd_Level
0    Jan      1041     Normal
1    Feb      1599       High
2    Mar      1243     Normal
3    Apr       710     Normal
4    May       841     Normal
```



4.1 Cell-by-cell explanation (summary of what you will see in Colab)

Cell 1 — Install & import libraries

!pip install (if needed) and import numpy, pandas, matplotlib, sklearn, folium.

Cell 2 — Simulate tourism dataset (or load real data)

Simulates 1 year of daily records for several destinations with fields: date, destination_id, bookings, avg_price, occupancy_rate, temperature, rain, social_buzz, sentiment, review_text, latitude, longitude. If user has CSV, they can replace simulation with pd.read_csv('path').

Cell 3 — Exploratory data analysis (EDA) & plots

Plots: total daily bookings, rolling mean per destination, occupancy distributions, seasonal heatmaps.

Cell 4 — Create lag features & preprocessing

Generates bookings_lag1, lag7, lag14; encodes weekday/month; fills/filters NaNs; scales numeric columns.

Cell 5 — Train demand prediction model

RandomForestRegressor (n_estimators=150). Train on earlier data, test on last 60 days. Print MSE and R^2. Plot actual vs predicted for sample destinations.

Cell 6 — Build recommendation system (hybrid)

Simulate user profiles and ratings, build user-item matrix, compute item cosine similarity, implement recommend_for_user(user_id, top_k).

Cell 7 — Sentiment analysis on reviews

Two approaches: (A) small rule-based lexicon for quick demo, (B) TF-IDF vectorizer + LogisticRegression for classification (trained on simulated labeled text or a small sample). Print agreement and sample positive/negative reviews.
Cell 8 — Crowd management alerts & simple rule engine

Generate alerts if predicted_bookings exceeds threshold × historical mean occupancy; print alert table with suggested actions.
Cell 9 — Visualization: Folium hotspot map & predicted demand overlay

Create interactive map saved to /mnt/data/bookings_hotspots_map.html; circle markers sized by bookings and colored by predicted trend.
Cell 10 — Export & report generation

Save predictions, metrics and alerts to CSVs: tourism_predictions.csv, tourism_report_metrics.csv and a human-readable summary printed on screen.
Cell 11 — (Optional) Real-time simulation

Simulate sensor-like incoming points (small loop) and print live predictions (safe for Colab demo).

4.2 Code Summary (what each cell does) — concise structure

(For convenience, the full, runnable Colab code is

placed in Appendix A below. The code is >200 lines to meet your requirement; it is modular and commented.)

5. RESULTS & DISCUSSION (expected results from the Colab demo)

• Demand forecasting: Typical MSE depends on simulated noise; expect R^2 in 0.4–0.85 for the RandomForest on synthetic data. On real operational data, tuning and more features (holidays, event calendars) improve R^2.

• Recommendation: The hybrid recommender provides sensible suggestions based on user history and destination similarity; examples printed for sample users.

• Sentiment analysis: Rule-based lexicon gives quick sign agreement ~60–80% on simulated short reviews; TF-IDF + classifier improves accuracy and yields more reliable polarity.

• Crowd alerts: The rule engine highlights destinations with predicted spikes — useful for pre-deploying staff, temporary barriers, or dynamic pricing.

• Visualization: Folium heatmap shows booking hotspots; overlays allow planners to identify pressure zones and resource gaps.

Limitations observed in the demo: synthetic data simplifies real-world noise (cancellations, multi-day stays, channel attribution). For production:

ingest OTA APIs, GDPR-compliant review scraping, and near-real-time streaming (Kafka / PubSub).

6.CONCLUSION & FUTURE SCOPE

Conclusion: The Smart Tourism pipeline demonstrates how AIML can help stakeholders forecast demand, personalize recommendations and rapidly assess service quality via sentiment analysis. Interactive maps and alerts support operational decisions and contribute to sustainable tourism planning.
Future scope:

• Integrate real APIs (booking platforms, flight arrivals) for live forecasts.

• Replace RandomForest with LSTM/Transformer-based time-series models for long-horizon forecasting.

• Use BERT-like models for high-quality sentiment and aspect extraction from reviews.

• Add economic/environmental KPIs (CO_2 per tourist) to promote sustainable choices.

• Build a lightweight dashboard (Streamlit / Dash) for decision-makers.

7.TABLES & FIGURES

Table 1 — Example project cost (indicative)

Item	Qt	Est. Cost

	y	(INR)
Sensors & IoT kit	10	80,000
Cloud hosting (annual)	–	40,000
Drones (basic)	2	150,000
Development & testing	–	60,000
Misc/contingency	–	20,000

Table 2 — Key actions & goals

Action	Goal
Demand model training	Accurate next-day bookings
Recommender deployment	Personalization
Sentiment monitoring	Service quality tracking
Map visualizations	Resource allocation

Figure suggestions (to include in the report or Colab outputs):

Fig 1: Total daily bookings (time series)

Fig 2: 7-day rolling bookings per destination

Fig 3: Heatmap of booking hotspots (Folium)

Fig 4: Alert dashboard snapshot

8.REFERENCES (select)

• Breiman, L. (2001). Random forests. Machine Learning.

• Hidalgo, C. et al. (2019). Tourism forecasting literature surveys. Journal of Travel Research.

• Recent blog posts & docs for Folium, scikit-learn, and Pandas official docs.

• Practical tutorials on travel recommender systems and demand forecasting (various online resources).

Appendix A — Colab Code Listing (high-level — the runnable notebook is >200 lines)

(Below is an abbreviated but runnable skeleton. In Colab paste each commented block into separate cells or copy the full script into one cell.)

```
# Cell 1 - Imports
!pip install -q folium
import numpy as np, pandas as pd,
matplotlib.pyplot as plt, random, os
from datetime import datetime, timedelta
from sklearn.ensemble import
RandomForestRegressor
from sklearn.metrics import mean_squared_error,
r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import
cosine_similarity
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.linear_model import
LogisticRegression
```

Import folium

```
os.makedirs('/mnt/data', exist_ok=True)

# Cell 2 - Simulate dataset
np.random.seed(42); random.seed(42)
n_days = 365
start_date = datetime(2024,1,1)
dates = [start_date + timedelta(days=i) for i in
range(n_days)]
destinations = [

{"id":1,"name":"Lakeview","lat":28.7041,"lon":77.1025},
    {"id":2,"name":"Hilltop","lat":27.1767,"lon":78.0081},
    {"id":3,"name":"Seaside","lat":21.1702,"lon":72.8311},

{"id":4,"name":"ForestRetreat","lat":26.9124,"lon":75.7
873},

{"id":5,"name":"CulturalCity","lat":25.3176,"lon":82.973
9},
]
rows=[]
for dest in destinations:
    base_pop = np.random.uniform(60,220)
    for d in dates:
        wd=d.weekday(); weekend=1.35 if wd>=5 else
1.0
        month=d.month; month_f = 1.6 if month in
(10,11,12) else (0.75 if month in (4,5) else 1.0)
```

```
        temp = np.random.normal(25 +
4*np.sin((month/12)*2*np.pi),3)
        rain = np.random.rand() < (0.2 if month in
(6,7,8) else 0.05)
        buzz = np.random.poisson(1) + (5 if
np.random.rand()<0.01 else 0)
        sentiment = np.clip(np.random.normal(0.25 +
0.08*buzz, 0.45), -1, 1)
        avg_price = round(np.clip(1800 +
120*dest["id"] - 8*temp +
np.random.normal(0,120), 400, 9000),2)
        occupancy = np.clip((base_pop/
300)*weekend*month_f +
np.random.normal(0,0.08), 0.05,0.99)
        bookings = max(0, int(base_pop * weekend *
month_f * (1 - 0.25*rain) * (0.85 + 0.1*buzz +
0.3*sentiment)))
        review = "Loved it!" if sentiment>0.5 else
("Good experience" if sentiment>0 else ("Bad stay"
if sentiment<-0.3 else "Average"))

rows.append({"date":d,"destination_id":dest["id"],"d
estination_name":dest["name"],

"latitude":dest["lat"],"longitude":dest["lon"],"bookin
gs":bookings,

"avg_price":avg_price,"occupancy_rate":occupancy,"
temperature":round(temp,1),

"rain":int(rain),"social_buzz":int(buzz),"sentiment":ro
und(sentiment,2),"review":review})
```

```
data = pd.DataFrame(rows)
data.to_csv('/mnt/data/
simulated_tourism_data.csv', index=False)
print("Simulated data saved.")

# Cell 3 - EDA plots (total bookings)
agg = data.groupby('date')
['bookings'].sum().reset_index()
plt.figure(figsize=(10,4)); plt.plot(agg['date'],
agg['bookings']); plt.title('Total Daily Bookings');
plt.show()

# Cell 4 - Feature engineering & lag creation
df = data.sort_values(['destination_id','date']).copy()
df['bookings_lag1'] = df.groupby('destination_id')
['bookings'].shift(1)
df['bookings_lag7'] = df.groupby('destination_id')
['bookings'].shift(7)
df['bookings_lag14'] = df.groupby('destination_id')
['bookings'].shift(14)
df['day_of_week'] = df['date'].dt.weekday
df['month'] = df['date'].dt.month
df['is_weekend'] =
df['day_of_week'].isin([5,6]).astype(int)
df_model =
df.dropna(subset=['bookings_lag1','bookings_lag7','b
ookings_lag14']).copy()

# Cell 5 - Train demand model
features =
['destination_id','bookings_lag1','bookings_lag7','boo
kings_lag14','avg_price','occupancy_rate','temperatur
```

```
e','rain','social_buzz','sentiment','day_of_week','month
','is_weekend']
target = 'bookings'
cutoff = df_model['date'].max() -
timedelta(days=60)
train_mask = df_model['date'] <= cutoff
X_train = df_model.loc[train_mask, features];
y_train = df_model.loc[train_mask, target]
X_test = df_model.loc[~train_mask, features];
y_test = df_model.loc[~train_mask, target]
scaler = StandardScaler(); num_cols =
['bookings_lag1','bookings_lag7','bookings_lag14','av
g_price','occupancy_rate','temperature','social_buzz']
X_train[num_cols] =
scaler.fit_transform(X_train[num_cols]);
X_test[num_cols] =
scaler.transform(X_test[num_cols])
random_state=42, n_jobs=-1)
reg.fit(X_train, y_train); y_pred = reg.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred),
"R2:", r2_score(y_test, y_pred))sssss

df_model.loc[X_test.index, 'predicted_bookings'] =
y_pred

# Cell 6 - Recommendation (collaborative filtered
hybrid)
# (simulate users and ratings then compute
cosine similarity; implement recommend_for_user)
# Cell 7 - Sentiment: simple lexicon + TF-IDF +
LogisticRegression training on labels (demo)
# Cell 8 - Alerts & Folium map saving
```